Relatório Técnico: SDR Imobiliário com IA Generativa

Projeto: SDR Imobiliário AI

Tipo: prova de conceito acadêmica

Data: outubro de 2026

Resumo executivo

O projeto implementa uma prova de conceito de atendimento imobiliário com a agente conversacional Bia. A solução reúne catálogo consultável, filtros determinísticos, recuperação híbrida de documentos, integração com Google Gemini, qualificação de leads, agendamento, follow-up e painel web. O objetivo é demonstrar um fluxo de atendimento integrado, não comprovar desempenho de produção.

O catálogo contém 248 registros sintéticos de imóveis em São Paulo. O sistema usa embeddings BAAI/bge-m3, um reranker opcional `cross-encoder/ms-marco-MiniLM-L-6-v2` e Gemini 2.5 Flash por padrão. A suíte contém 45 funções de teste declaradas no código, mas não há benchmark registrado de qualidade de recuperação, classificação, latência ou custo. Portanto, este relatório não atribui métricas de qualidade não medidas.

Problema escolhido e objetivo

O problema é apoiar o primeiro atendimento de potenciais compradores, locatários e investidores, reduzindo a coleta manual de requisitos e organizando a passagem do lead para o corretor. Uma resposta generativa sem evidência do catálogo pode inventar imóveis ou características; por isso, o agente dispõe de ferramentas que consultam o catálogo e o prompt instrui a Bia a não inventar informações.

O fluxo contempla descoberta da intenção e das preferências, busca por filtros ou descrição livre, registro dos dados do lead, classificação de temperatura, sugestão de horários e follow-up. A interface é uma aplicação React; a API é construída com FastAPI.

Dados utilizados

A fonte é `backend/data/imoveis.json`, uma base simulada e mantida no próprio repositório. A fotografia do arquivo analisada para este relatório tem 248 imóveis: 129 para compra e 119 para aluguel. Há 40 bairros distintos e seis tipos: 91 apartamentos, 45 casas, 42 studios, 25 lofts, 23 sobrados e 22 coberturas.

Os registros incluem identificador, título, tipo, finalidade, preço, cidade, bairro, quantidade de quartos, banheiros e vagas, área em metros quadrados, descrição e disponibilidade. Os dados não representam um inventário imobiliário real nem uma amostra estatisticamente representativa.

Preparação dos dados

O repositório carrega o JSON e valida cada item com o modelo Pydantic `Imovel`. A busca estruturada compara campos como finalidade, tipo, cidade, bairro, preço máximo ou mínimo, quartos mínimos e vagas mínimas; não há treinamento de modelo para esses filtros.

Para a recuperação textual, cada imóvel é convertido em um documento com título, tipo, finalidade, cidade, bairro, preço, quartos, banheiros, vagas, área e descrição. Os mesmos dados-chave são guardados como metadados. O texto usado para correspondência lexical é normalizado com minúsculas, remoção de diacríticos e pontuação e consolidação de espaços. Não há etapa de deduplicação, anotação humana, divisão treino/validação/teste ou enriquecimento com fonte externa descrita no pipeline.

O script `backend/scripts/indexar_imoveis.py` grava os documentos em uma coleção Chroma chamada `imoveis`, usando IDs `imovel-{id}`. A validação opcional do script é uma consulta de fumaça para imóveis familiares com pelo menos três quartos; ela não calcula uma métrica de relevância.

Modelos e métodos

Filtros estruturados: lógica determinística em `CatalogoService`, sem modelo aprendido.

Embeddings: `BAAI/bge-m3`, carregado por `HuggingFaceEmbeddings`, com embeddings normalizados. O código compara produto interno entre o vetor da consulta e os vetores dos documentos.

Reranker: `cross-encoder/ms-marco-MiniLM-L-6-v2`, carregado de forma opcional. Quando disponível, a pontuação combina 45% similaridade vetorial, 25% pontuação lexical e 30% reranker. Quando indisponível, a combinação passa a 70% vetorial e 30% lexical. A falha de carregamento é capturada e o sistema prossegue sem reranker.

LLM: Google Gemini, via SDK `google-genai`, com `gemini-2.5-flash` como padrão configurável por `GEMINI_MODEL`. O modelo seleciona ferramentas de busca estruturada ou semântica e pode chamar ferramentas adicionais de qualificação e agenda. O código limita a dez rodadas de chamadas de ferramenta por mensagem. Não há ajuste fino do LLM descrito no projeto.

Esses componentes estão configurados e exercitados por testes funcionais; isso não equivale a um estudo comparativo entre modelos. Não foram encontrados resultados experimentais comparando embeddings, rerankers ou LLMs alternativos.

Funcionamento do RAG e integração com a LLM

1. A consulta chega à API ou ao agente de chat.

2. Para critérios explícitos, o agente pode chamar `buscar_imoveis_estruturado`; para solicitações descritivas, pode chamar `buscar_imoveis_semantico`.

3. A busca semântica aplica filtros, quando informados, calcula similaridade vetorial e pontuação lexical, e opcionalmente reordena os resultados com o cross-encoder. A ordenação final limita os resultados ao `limite` solicitado, que é cinco por padrão na ferramenta do agente.

4. Os imóveis recuperados são devolvidos ao Gemini como resposta da ferramenta. O modelo redige a resposta conversacional sob instruções para não afirmar características que não estejam no catálogo.

Há uma diferença importante entre indexação e consulta na implementação atual: a indexação grava vetores no Chroma, porém `ImovelRAG.buscar()` percorre os documentos gerados a partir do JSON e calcula novamente os embeddings dos candidatos para cada consulta. O caminho de consulta não chama uma busca de similaridade no `vector_store`. Assim, o Chroma é persistido, mas não acelera a recuperação semântica atualmente.

As sessões de conversa da Bia são mantidas em memória do processo. Os dados de lead e agenda também são mantidos nos repositórios em memória da prova de conceito; reiniciar o processo não oferece persistência transacional.

Métricas e resultados disponíveis

Os números objetivos disponíveis descrevem o conjunto de dados e a configuração, não a qualidade da IA:

- 248 imóveis cadastrados: 129 para compra e 119 para aluguel.
- 40 bairros e seis categorias de tipo de imóvel.
- 45 funções de teste declaradas em `backend/tests/`, cobrindo filtros, regras de qualificação, integração, agenda, follow-up e comportamento do agente com respostas simuladas.
- Na execução realizada para esta entrega: 47 testes passaram em 1,04 s; houve um aviso de depreciação relacionado ao alias `anyio.abc.BlockingPortal` usado pelo `starlette.testclient`.
- A classificação de leads é uma regra explicável de 0 a 100 pontos. Os limiares no código são quente a partir de 70 e morno a partir de 40; abaixo de 40 é frio.

Os testes são funcionais e não calculam métricas de precisão, recall, MRR, nDCG, taxa de alucinação, qualidade da classificação, sucesso de agendamento, latência, tokens ou custo. Também não há conjunto de consultas anotado ou baseline. Esses itens devem ser medidos antes de qualquer alegação de eficácia ou prontidão para produção.

Limitações e riscos

- Dados sintéticos, pequenos e limitados a uma única cidade; não refletem disponibilidade, preços ou linguagem de clientes reais.
- Ausência de avaliação offline e de avaliação humana das respostas, da relevância do ranking e da qualidade de qualificação.
- A busca semântica re-embeda todos os candidatos a cada consulta e não usa o índice persistido do Chroma na recuperação; isso pode elevar latência e consumo de memória.
- Regras lexicais específicas para alguns termos podem favorecer consultas previstas e falhar em paráfrases ou novas categorias.
- O reranker é opcional e pode falhar silenciosamente, alterando a fórmula de ranking sem telemetria explícita.
- O Gemini depende de rede, chave, cota e serviço externo. Custo, privacidade, retenção de dados e comportamento em indisponibilidade precisam de política operacional.
- Sessões, leads e agenda em memória não oferecem persistência, coordenação entre réplicas, autenticação, autorização ou trilha de auditoria de produção.
- O prompt reduz o risco de afirmações sem suporte, mas não substitui validação de saída nem avaliação de alucinações.

Próximos passos de avaliação

Construir um conjunto de consultas imobiliárias representativas, com filtros e documentos relevantes julgados por pessoas; comparar filtros, BGE-M3 sem reranker e BGE-M3 com reranker usando Recall@k, Precision@k, MRR e nDCG; executar os testes de integração com LLM simulada e manter avaliações separadas com respostas do Gemini; medir latência, uso de tokens e custo; e registrar resultados, versões de modelos e configuração para que os experimentos sejam reproduzíveis. Para produção, persistir leads e sessões em banco de dados e instrumentar falhas de modelo e de ferramentas.

Execução containerizada

O `Dockerfile` na raiz tem alvos para FastAPI e frontend estático. O `docker-compose.yml` constrói e inicia ambos; o Nginx serve o React e encaminha `/api` ao backend. O índice Chroma e o cache de modelos usam volumes persistentes. A primeira inicialização do backend indexa o catálogo e pode baixar o modelo de embeddings, portanto pode demorar e requer acesso à internet.

Com Docker Compose instalado, execute na raiz do repositório:

`sh

docker compose up --build

O frontend fica em `http://localhost:5173` e a documentação da API em `http://localhost:8000/docs`. Para habilitar chat e funções generativas, configure `backend/.env` com `GEMINI_API_KEY` e, opcionalmente, `GEMINI_MODEL`. Sem a chave, catálogo e interface podem iniciar, mas o chat depende da API Gemini.